

token-burner 竞品基准分析

版本基准: v0.4.0-alpha (2026-08-31 交付) | 更新范围: 纳入 v0.4 已交付能力与 v0.5 (Beta) 规划 | 数据说明: 基于公开资料与产品规格的定性对比, 评分不代表第三方评测结论

一、执行摘要

token-burner 在 v0.4 (Alpha) 完成从「代码补全助手」到「项目级多智能体交付系统」的关键跨越: 容器级安全沙箱 (只读文件系统 / 无网络 / 非 root / 资源配额)、异步任务 API 与 SSE 实时事件流 (4 路并发、任务状态落盘可恢复)、双模式意图识别 (System-1 快判 + System-2 全量评估, 闲聊/基础类诉求不再消耗全量评估成本)、三档智能模型路由、Embedding / 论点库 / _shared 三级缓存、VS Code 插件全流程闭环、六层成本护栏与全链路可审计。上述组合能力在同类竞品中**尚无完整对标**——多数竞品强于单点交互体验, 弱于成本治理与可审计交付。

二、竞品能力矩阵 (2026-08 定性基准)

能力维度	token-burner v0.4	Cursor	GitHub Copilot Workspace	Devin (Cognition)	OpenHands	通义灵码 / 文心快码
任务层级: 项目级自主交付	✓ 模块级拆分 + 团队流程	● 会话级	● Issue 级	✓ 端到端	✓ 端到端	✗ 补全/问答
多智能体协作 (PM/评审/Dev/Test 分角色)	✓ 三模型互异团队 + 双评审收敛	✗ 单体	✗ 单体	● 内部规划器	● 可配置角色	✗
方案讨论收敛机制 (双评审 + 收敛上限)	✓	✗	✗	●	●	✗

能力维度	token-burner v0.4	Cursor	GitHub Copilot Workspace	Devin (Cognition)	OpenHands	通义灵 码 / 文心快 码
成本护栏（六层：预算闸门/截断/去重/修复上限/收敛上限/看板）	✓ 全量	✗ 订阅制	✗ 订阅制	✗ 按任务计费	✗	✗
智能模型路由（按难度三档降档）	✓ (v0.4)	🟡 手动切换	🟡 固定组合	🟡	🟡	🟡
上下文缓存（Embedding / 论点库 / _shared）	✓ (v0.4)	🟡 内置索引	🟡	✗	✗	🟡
安全沙箱：容器级隔离（只读 fs / 无网 / 非 root）	✓ (v0.4)	✗ 本地进程	🟡 云 Codespace	✓ 云 VM	✓ Docker	✗
资源配额（CPU / 内存 / pids / 磁盘）	✓ (v0.4)	✗	🟡	🟡	🟡	✗
危险操作预扫描（AST 黑名单，执行前拦截）	✓	✗	✗	🟡	🟡	✗
异步任务 API（提交即返回 + 状态落盘可恢复）	✓ (v0.4)	✗ 交互式	🟡 Action 运行	✓ 会话制	🟡	✗
SSE 实时进度事件流	✓ (v0.4)	✗	🟡	✓	🟡	✗
中断恢复（断点续跑，已完成模块跳过）	✓ 磁盘快照重建	✗	✗	🟡 会话保留	🟡	✗

能力维度	token-burner v0.4	Cursor	GitHub Copilot Workspace	Devin (Cognition)	OpenHands	通义灵 码 / 文心快 码
IDE 集成 (VS Code: 发起/ 进度/应用代 码/成本)	✓ (v0.4)	✓ 原生	✓	✗ Web	✗ Web	✓
全链路可审计 (决策记录 / 校验结果 / 消 耗明细落盘)	✓	✗	●	●	●	✗
本地 / 私有化 部署 (数据不 出域)	✓ 本地运行	✗ 云	✗ 云	✗ 云	✓ 开源自托管	● 混 合
成本模型 (自 带 Key, 按用 量付费)	✓ BYOK	✗ 订阅	✗ 订阅	✗ 订阅/计费	✓ BYOK	● 混 合
中文场景与国 内模型生态	✓	●	●	✗	●	✓

口径说明：「✓」为产品内建的系统性能力；「●」为部分具备或依赖外部平台；「✗」为公开资料未见。竞品能力随版本迭代变化，本表仅作产品定位参考，不构成质量评价。

三、v0.4 交付后的差异化定位

3.1 三条独有护城河

- **成本确定性**：唯一把「六层成本护栏 + 三级缓存 + 智能路由 + 节省量看板」作为一等公民的产品——竞品的成本控制普遍停留在订阅定价层面，缺少任务内的程序化闸门。
- **可审计交付**：模块级决策记录、接口契约门禁、修复轮数上限、成本报告全部落盘可回溯——直接对应企业采购的安全与审计诉求。
- **过程质量工程**：PM/双评审/收敛的「团队讨论」阶段在竞品中普遍缺失；该阶段把「需求→方案」的失配在写码前拦截，是交付质量的结构性的保障。

3.2 相对短板（诚实盘点）

- 交互顺滑度不及 Cursor（逐行补全/内联编辑是 Cursor 核心场景，token-burner 定位为项目级交付，二者互补而非替代）。
- 云原生体验不及 Devin（全托管、零环境搭建）；token-burner 的本地/私有化定位是刻意的安全取舍。
- 生态品牌与分发渠道弱于 GitHub Copilot。

四、v0.5（Beta）规划对竞争格局的影响

v0.5 规划能力	竞品现状	交付后的竞争意义
Researcher Agent (文档调研 + 知识注入，独立预算与缓存)	Devin/Cursor 均无结构化「调研→注入→可缓存」链路；OpenHands 支持浏览器但无预算治理	解决 LLM 知识过时导致的技术方案不可行问题；调研结果按「技术栈 + API + 版本」缓存复用，竞品无对标
实时监控面板升级 + Agent 对话流图	LangSmith/LangFuse 等观测工具需独立接入	把「多智能体黑盒」可视化为本产品自带能力，降低企业信任门槛
模式智能推荐（按历史成功率/成本自动选 safe 或 auto）	竞品无模式概念	双模式（安全审阅 / 自动验证）是本产品独有治理模型，推荐引擎进一步降低使用门槛
P2 顺延并入：任务取消与僵尸清理、镜像预热、Docker 桌面引导、自动发布流水线	Devin 具备任务取消；其余普遍缺失	补齐企业级运维闭环的最后短板

五、结论

v0.4 交付后，token-burner 在「成本治理 × 可审计交付 × 项目级多智能体」三个维度形成**结构性差异化**，与 Cursor（交互体验）、Devin（全托管云）、通义灵码（国内生态）不在同一正面战场。v0.5 的 Researcher 与可视化能力将进一步拉开「过程可观测性」差距；主要竞争风险仍来自 GitHub/Cursor 生态位下沉（从补全向项目级任务演进）与 Devin 的降价扩张——应对策略是持续放大「本地私有化 + BYOK + 可审计」的企业侧不可替代性，并以 v0.5 智能化能力巩固先行者优势。